

技术报告-周舟粥

长上下文ICL标注任务技术报告

- 1. 方案概述
 - 核心创新点
 - 技术架构
- 2. 方法详述
 - 2.1 稳定性优化
 - 2.2 cleanup专项优化
 - 2.3 上下文长度策略
- 3. 实验结果
 - 3.1 数据集
 - 3.2 性能指标
 - 3.3 关键改进
- 4. 技术创新
 - 4.1 多层兜底机制
 - 4.2 智能代码清洗
- 5. 复现说明
 - 5.1 环境要求
 - 5.2 运行命令
 - 5.3 预期运行时间
- 6. 总结

长上下文ICL标注任务技术报告

1. 方案概述

本方案基于cleanup优化策略，针对长上下文ICL标注任务进行了专项优化。

核心创新点

- 稳定性增强：修复了Task 8的思维链标签过滤问题，实现零null输出
- 动态检索：从按任务固定示例改为按样本动态选择最相关示例
- 专项优化：针对不同任务类型设计了专门的优化策略

技术架构

- 基础模型：Qwen3-4B
- 推理框架：FlagScale + vLLM
- 硬件环境：Ascend 910C x2

2. 方法详述

2.1 稳定性优化

- 修复了annotate_ascend函数中Task 8的思维链标签过滤正则表达式
- 增强了count_answer函数，添加了5层兜底解析策略
- 强化了build_prompt的输出格式约束

2.2 cleanup专项优化

- Task 8去思维链清洗：过滤思维链标签，优先提取代码块
- 代码结构识别：自动识别import、def、@triton.jit等代码标记

2.3 上下文长度策略

- Task 1-7：前50样本使用30k上下文，其余使用8k上下文
- Task 8：固定使用16k上下文（官方要求）

3. 实验结果

3.1 数据集

- 8个评测任务，共3666个测试样本
- 涵盖数学推理、字符串处理、情感分析、代码生成等多种任务

3.2 性能指标

任务	样本数	完成率	预估提升
Task 1	500	100%	+1~3%
Task 2	500	100%	+1~3%
Task 3	500	100%	+1~3%
Task 4	500	100%	+1~3%
Task 5	500	100%	+0.5~2%
Task 6	500	100%	+0.5~2%
Task 7	500	100%	+0.5~2%
Task 8	166	100%	+1~3%

3.3 关键改进

- Task 8 null率从90%+降低至0%
- 整体稳定性显著提升
- 代码生成质量明显改善

4. 技术创新

4.1 多层兜底机制

设计了5层解析兜底策略，确保在任何情况下都能返回有效结果：

1. Label标签解析
2. 引号内容提取
3. 数字提取
4. 常见标签匹配
5. 文本截取兜底

4.2 智能代码清洗

针对Task 8设计了专门的代码清洗逻辑：

- 思维链标签过滤
- 代码块优先提取
- 代码结构识别

5. 复现说明

5.1 环境要求

- Python 3.11+
- PyTorch 2.0+
- vLLM 0.4+
- FlagScale框架

5.2 运行命令

```
1 cd code
2 export SELECT_METHOD="select_examples_M12_cleanup"
3 for task_id in {1..8}; do
4     python main.py --task_id $task_id --tokenizer_path /root/Qwen3-4B --log
      _path_prefix ../outputs/
5 done
```

5.3 预期运行时间

- 串行：约4.2–4.4小时
- 4任务并行：约1.4–1.5小时

6. 总结

cleanup方案通过稳定性修复和专项优化，显著提升了长上下文ICL标注任务的可靠性。特别是在Task 8代码生成任务中，实现了从高null率到零null的突破性改进。

本方案的技术创新点在于：

1. 系统性的稳定性保障机制

2. 针对性的任务优化策略

3. 工程化的兜底处理方案

这些改进为长上下文ICL技术的实际应用提供了重要的技术基础。